VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

IoT MULTIDISCIPLINARY PROJECT

Project

Smart Toll Gate System

Instructor(s): Võ Thị Ngọc Châu

Students: Phùng Gia Huy - 2352406 *(Group CC01 - Team 05)*
Đặng Quốc Khánh - 2352515 *(Group CC01 - Team 05, **Leader**)*
Ngô Xuân Kiên - 2352642 *(Group CC01 - Team 05)*
Trần Ngọc Thoại - 2353147 *(Group CC01 - Team 05)*

HO CHI MINH CITY, APRIL 2026

Contents

Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Phùng Gia Huy	2352406	- Hardware Integration - IoT Gateway Implementation	100%
2	Đặng Quốc Khánh	2352515	- Database Design - Backend Development	100%
3	Ngô Xuân Kiên	2352642	- Hardware Integration - Embedded Systems	100%
4	Trần Ngọc Thoại	2353147	- Artificial Intelligence (AI) - Frontend Development	100%

Table 1: Member list & workload

1 System Overview

1.1 System Description

The **Smart Toll Gate System** is an intelligent vehicle access control solution designed for residential areas and apartment complexes. The system integrates IoT devices, Artificial Intelligence, and modern web technologies to automate the process of identifying vehicles and controlling gate barriers.

Key Features:

- **Automatic License Plate Recognition:** Uses AI-powered camera system to detect and recognize vehicle license plates in real-time.
- **Intelligent Access Control:** Automatically opens barriers for registered vehicles and alerts security guards for unknown vehicles.
- **Guest Management:** Allows residents to pre-register guests or generate OTP codes for unexpected visitors.
- **Real-time Monitoring:** Provides security guards with a dashboard to monitor all gate activities and perform manual overrides when needed.
- **Comprehensive Logging:** Records all access events with timestamps, images, and AI confidence scores for audit purposes.

1.2 System Context Diagram

The following diagram illustrates the Smart Toll Gate System in its real-world context, showing the interactions between the system and external entities (actors).

image/system_context.png

Figure 1: System Context Diagram - Smart Toll Gate System in Real-World Context

External Entities:

- **Resident Vehicles:** Cars and motorcycles owned by residents that need automatic access.
- **Guest Vehicles:** Visitors' vehicles that require pre-registration or OTP verification.
- **Citizens (Residents):** House owners who register vehicles, manage guests, and generate OTP codes.
- **Security Guards:** Personnel who monitor the dashboard and perform manual verifications.
- **Managers:** Administrators who approve registrations and view analytics.
- **IP Camera:** Hardware device that captures vehicle images at the gate, which also reads dynamic QR codes from pedestrian smartphones directly via AI OCR libraries.

- **Arduino + Servo Motor:** Hardware that physically controls the gate barrier.

1.3 Overall Architecture

The system follows a **Three-Tier Architecture** with additional IoT and AI components. The architecture separates concerns into distinct layers for maintainability and scalability.

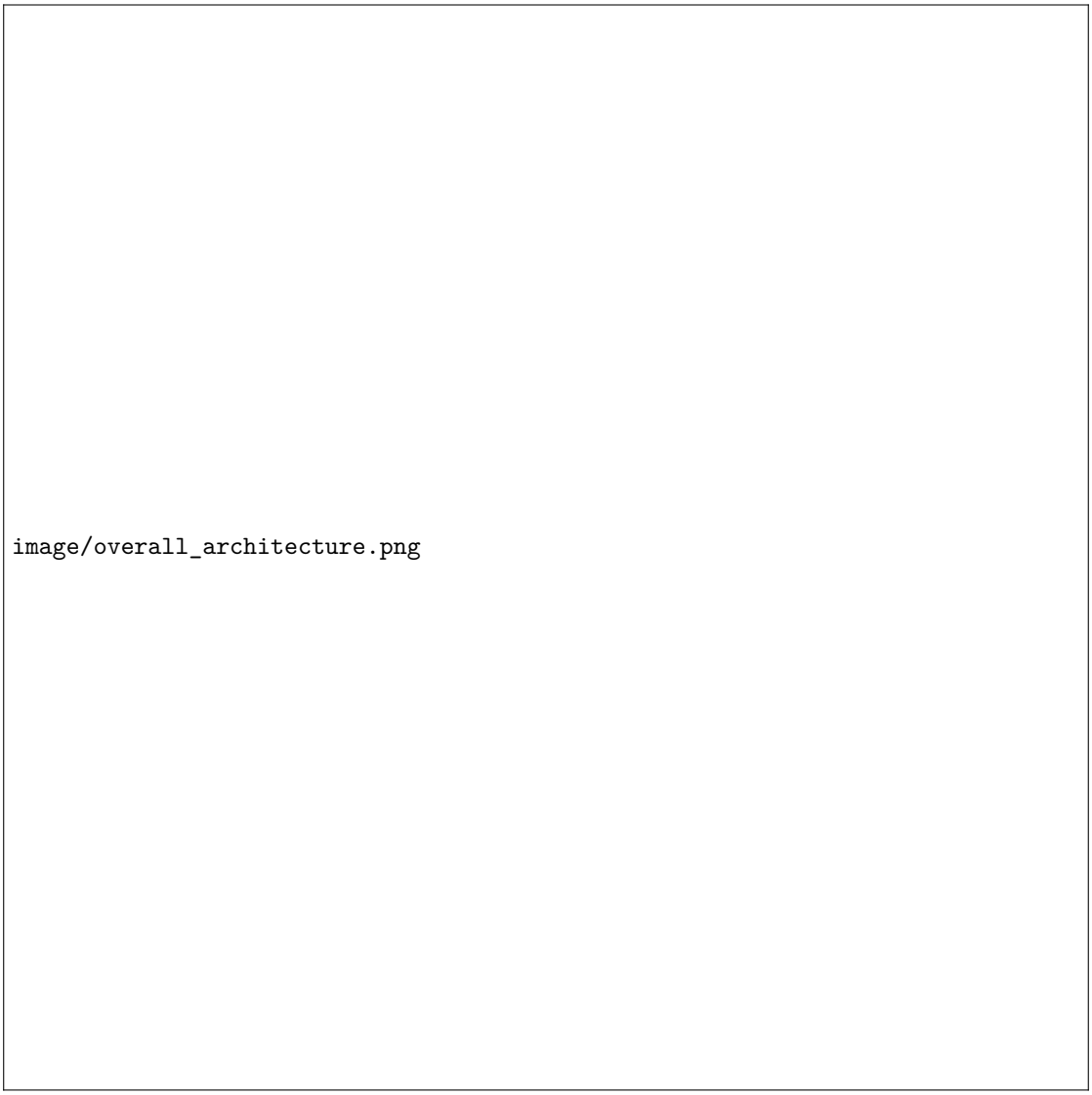

Figure 2: Overall System Architecture

Architecture Components:

- **Presentation Layer:** React.js web application with Material UI for citizen, guard, and manager dashboards.
- **Application Layer:** Node.js/Express backend server handling business logic, authentication, and API endpoints.

- **Data Layer:** PostgreSQL database storing users, vehicles, access logs, and images.
- **AI Service:** Python-based service using YOLOv8 for plate detection and PaddleOCR for text extraction, and OCR to read OTP/QR codes displayed on smartphone screens at the gate.
- **IoT Gateway:** Python application bridging the backend with Arduino hardware for gate control.
- **Hardware Layer:** Arduino, servo motor, IR sensors, and IP camera.

2 Overall Project Requirements

2.1 Project Scope

The Smart Toll Gate System project encompasses the following scope:

- Development of a complete vehicle access control system for residential areas.
- Integration of AI-based license plate recognition with IoT hardware control.
- Implementation of role-based web dashboards for different user types.
- Design and implementation of a relational database for data persistence.
- Development of at least 5 functional modules as specified below.

2.2 System Modules

The project is divided into the following **6 main modules**:

Table 2: System Modules

No.	Module Name	Description
1	AI Recognition Module	Implements YOLOv8 for license plate detection and PaddleOCR for text extraction. Provides REST API for image processing.
2	IoT Gateway Module	Python application that communicates with Arduino via serial port, captures images from IP camera, and coordinates with backend API.
3	Backend API Module	Node.js/Express server handling authentication, vehicle verification, access logging, and communication with all other modules.
4	Frontend Dashboard Module	React.js web application with role-based interfaces for citizens, security guards, and managers.
5	Database Module	PostgreSQL database design and implementation including schema, indexes, stored procedures, and data integrity constraints.
6	Hardware Control Module	Arduino firmware for reading IR sensors, controlling servo motor, and serial communication with IoT Gateway.

2.3 Technology Stack

Table 3: Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Vite + Tailwind CSS	Building responsive, role-based user interfaces

Table 3: ... (continued)

Layer	Technology	Purpose
Backend	Node.js + Express.js	RESTful API server with JWT authentication
Database	PostgreSQL	Relational database for persistent storage
AI Service	Python + YOLOv8 + PaddleOCR	License plate detection and text recognition
IoT Gateway	Python + python-socketio + OpenCV + PySerial	Bridge between backend and hardware via WebSocket
Hardware	Arduino + Servo Motor	Physical gate barrier control
Communication	HTTP/REST + WebSocket (Socket.IO) + Serial	Inter-module communication protocols

2.3.1 Hardware Components

MCU (Microcontroller Unit): Arduino Uno R3 - controls barrier, lights, and buzzer.

Sensors/Actuators:

- **Servo MG90S:** Simulates barrier gate mechanism.
- **Camera:** Captures license plate images as input.
- **LCD1602:** Displays notifications and status at the gate.
- **Ultrasonic Sensor HC-SR04:** Detects approaching vehicles.

Input/Output Classification:

Table 4: Hardware Input/Output Components

Type	Component	Function
Input	Camera	Captures license plate images
Input	Ultrasonic Sensor HC-SR04	Detects vehicle presence and proximity
Output	Servo MG90S (Barrier)	Controls barrier open/close mechanism
Output	LCD1602	Displays information (plate number, status)

Component List:

Table 5: Hardware Component List

No.	Component	Quantity	Description
1	Arduino Uno R3	1	Main microcontroller board

2	Servo MG90S	1	Micro servo motor for barrier control
3	LCD1602	1	16x2 character LCD display
4	Ultrasonic Sensor HC-SR04	1	Distance sensing for vehicle detection

2.3.2 Technology Comparison

1/ AI/IoT Programming Language Comparison

For the AI Recognition Module and IoT Gateway, we compared several programming languages:

Table 6: AI/IoT Programming Language Comparison

Language	Advantages	Disadvantages
Python	• Extensive AI/ML libraries (TensorFlow, PyTorch, YOLOv8) • Easy to integrate with OpenCV for image processing • Simple syntax, rapid development • Strong community support for IoT	• Slower execution compared to compiled languages • Higher memory consumption
C++	• Very fast execution speed • Low memory footprint • Direct hardware access	• Complex syntax, longer development time • Limited AI/ML library ecosystem • Manual memory management
Java	• Platform independent (JVM) • Good for enterprise applications • Strong type system	• Limited AI/ML support compared to Python • Verbose syntax • Higher resource consumption

Conclusion: We selected **Python** for the AI and IoT modules due to its extensive machine learning libraries (YOLOv8, PaddleOCR), easy integration with OpenCV for image processing, and rapid development capabilities essential for prototyping IoT applications.

2/ Frontend Framework Comparison

For the Frontend Dashboard Module, we compared popular JavaScript frameworks:

Table 7: Frontend Framework Comparison

Framework	Advantages	Disadvantages
React.js	• Component-based architecture • Large ecosystem (Material UI, Redux) • Virtual DOM for performance • Strong community and job market	• Requires additional libraries for routing/state • JSX learning curve
Angular	• Full-featured MVC framework • Built-in routing and forms • TypeScript by default	• Steep learning curve • Heavier bundle size • More opinionated structure
Vue.js	• Easy to learn • Lightweight and fast • Good documentation	• Smaller ecosystem • Less enterprise adoption • Fewer job opportunities

Conclusion: We selected **React.js** for the frontend because of its component-based architecture that supports reusable UI elements, extensive ecosystem with Material UI for professional styling, and strong community support. React's flexibility allows us to easily implement role-based dashboards for citizens, guards, and managers.

3/ Backend Framework Comparison

For the Backend API Module, we compared popular server-side technologies:

Table 8: Backend Framework Comparison

Technology	Advantages	Disadvantages
Node.js + Express	• Non-blocking, event-driven architecture • Same language (JavaScript) as frontend • Excellent for real-time applications • Large npm ecosystem • Fast development cycle	• Single-threaded (CPU-intensive tasks) • Callback complexity (mitigated by async/await)
Python + Django	• Batteries-included framework • Built-in admin interface • Strong ORM support • Good for rapid prototyping	• Slower than Node.js for I/O operations • Monolithic architecture • Steeper learning curve for Django

Java + Spring Boot	• Enterprise-grade stability • Strong type safety • Excellent for large-scale systems • Comprehensive security features	• Verbose boilerplate code • Slower development speed • Higher memory consumption • Complex configuration
PHP + Laravel	• Easy to learn and deploy • Good hosting support • Elegant syntax	• Performance limitations • Less suitable for real-time apps • Inconsistent core functions

Conclusion: We selected **Node.js with Express.js** for the backend because of its:

- **Unified JavaScript Stack:** Using JavaScript for both frontend (React.js) and backend simplifies development, reduces context switching, and allows code sharing.
- **Event-Driven Architecture:** Perfect for handling multiple concurrent connections from IoT devices, mobile apps, and web dashboards simultaneously.
- **Real-Time Capabilities:** Native support for WebSocket and real-time event streaming, essential for live gate status updates and notifications.
- **Lightweight and Fast:** Ideal for RESTful API development with minimal overhead, resulting in quick response times for vehicle verification.
- **Rich Ecosystem:** NPM provides packages like JWT for authentication, bcrypt for password hashing, and node-postgres for database connectivity.

2.4 Role-Based Access Control (RBAC)

The system implements Role-Based Access Control to ensure proper authorization for different user types. Each role has specific permissions and access to different features.

Table 9: Role-Based Access Control (RBAC) Matrix

Feature / Permission	**Manager**	**Security Guard**	**Citizen**
View Analytics Dashboard	✓(Full)	✗	✗
View Real-time Access Events	✓(Read-only)	✓(Full)	✗
Manual Gate Control	✓(Remote)	✓(Full)	✗
Vehicle Management	✓(All users)	✗	✓(Own)
Guest Registration	✓(Full)	✓(Full)	✓(Full)
Generate OTP Codes	✗	✗	✓(Own guests)
Verify OTP Codes	✗	✓(Full)	✗
User Management	✓(Full)	✗	✗
View Access Logs	✓(All)	✓(Own gate)	✓(Own vehicles)

Table 9: Role-Based Access Control (RBAC) Matrix (continued)

Feature / Permission	**Manager**	**Security Guard**	**Citizen**
System Settings	✓(Full)	✗	✗
Approve Vehicle Registration	✓(Full)	✗	✗

3 Business Requirements

3.1 Business Context

Modern residential areas and apartment complexes face several challenges:

- **High Labor Cost:** Need security guards to work 24/7 at each gate.
- **Slow Manual Verification:** Causes traffic jams during rush hours.
- **Lack of Transparency:** No evidence or history of vehicles entering or leaving.
- **Difficult Guest Management:** Cannot control unknown vehicles properly.

3.2 Business Goals

Table 10: Business Goals

ID	Business Goal	Success Metric
BG-01	Reduce average check-in time	Less than 3 seconds per vehicle (with AI)
BG-02	Optimize operational costs	Reduce 50% security staff needed
BG-03	Improve area security	100% log entries with image evidence
BG-04	Improve resident experience	Satisfaction rate greater than 90%
BG-05	Data-driven decision making	Real-time analytics dashboard

3.3 Business Scenarios

Scenario 1: Happy Path (Fully Automated Flow & Anti-spoofing)

Step 1: Scan Permanent Whitelist (vehicles table)
- The Backend prioritizes checking if this is a resident's vehicle. It queries the Database for a vehicle where `license_plate = AI_plate` and `is_active = true`.
- If NOT FOUND: The vehicle does not belong to a resident → Immediately move to **Step 2**.
- If FOUND: The system starts the security check:
  - **Anti-Passback Check:** The system verifies the vehicle's direction (inbound/outbound) against its state in the database. If it is already `is_inside = true` and going inbound, or `is_inside = false` and going outbound → Logic error → Block immediately, alert anti-passback violation.
- **Approval:** If the security test passes → Toggle `is_inside` flag → Save log → Open the gate. Flow ends.

Step 2: Scan Temporary Whitelist (guest_registrations table)
- If the vehicle fails Step 1 (not in the `vehicles` table), the Backend automatically searches the pre-registered guest list.
- It queries the Database for `guest_license_plate = AI_plate` and `status = 'approved'`.
- **Time Window Validation:** The system compares the current time against the guest's allowed window (`visit_start_time` $\leq$ current time $\leq$ `visit_end_time`).
- If FOUND and VALID TIME: This is an authorized guest → Save log (storing guest ID) → Open the gate. Flow ends.
- If NOT FOUND or OUTSIDE TIME WINDOW: The guest is using the wrong vehicle, arrived too early, or the permit expired → Access denied. Move to **Step 3**.

Step 3: Fallback (Deny & Redirect)
- If the vehicle fails both Step 1 and Step 2:
- The Backend logs `is_access_granted = false` (Access denied) and sends a `KEEP_CLOSED` command to the IoT Gateway.
- A real-time notification is pushed to the Security Guard dashboard: 'Invalid vehicle. Please handle!"
- The guest must now switch to Scenario 3: Call the resident to obtain a 6-digit OTP and read it to the Guard.

Scenario 2: Manual Override (Remote Human Intervention)

1. This flow is used for special cases that require visual confirmation by the Security Guard or Manager.
2. Priority vehicles (ambulances, fire trucks, police) or exceptional cases enter the lane.
3. The Security Guard or Manager monitors the live camera feed on their respective dashboard.
4. The Guard or Manager clicks the Emergency Gate Open button on their web interface to remotely open the gate.
5. The system requires the operator to select a reason (e.g., emergency_vehicle, system_maintenance).
6. The backend stores the action log together with the operator ID and sends the gate-open command.

Scenario 3: Automated Guest Access via Camera OCR (OTP)

1. An unexpected guest (walk-in) or delivery driver arrives at the gate and is blocked because their plate is not registered.
2. The guest calls the resident to inform them of their arrival.
3. The resident opens the Mobile App, selects "Create OTP" — the system generates a random 6-digit code (valid for 15 minutes) and displays it on the app.
4. The resident sends the OTP code to the guest via phone call or message.
5. [**Fully Automated — No Guard interaction required**] The guest simply holds their smartphone screen showing the 6-digit OTP towards the Camera at the gate lane.
6. The AI Service analyzes the camera frame: detects the bright phone screen, and uses OCR to extract the 6-digit string.
7. The extracted code is sent to the Backend (`POST /api/v1/gates/verify-camera-otp`). Backend validates: code matches an active OTP, not used, not expired → marks `is_used = true` → logs access → sends `OPEN` command. Gate opens automatically.
8. The Security Guard sees the log appear on the real-time Dashboard (host name, guest info, timestamp).

Scenario 4: Pedestrian/Bicycle Access (QR-based)

1. A resident goes for a walk or rides a bicycle, which does not have a license plate for AI recognition.
2. The resident opens the Mobile App and generates a personal dynamic QR code (containing a UUID, valid for 3 minutes).
3. The resident holds their smartphone screen towards the AI Camera.
4. The AI Camera detects and decodes the QR code via OCR, sending the UUID string along with the snapshot to the Backend. The Backend verifies the token data → Opens the gate.

4 System Requirements

4.1 Functional Requirements

Table 11: Functional Requirements

ID	Requirement	Feasible	Testable	Important
FR-01	The system shall process AI recognition (YOLOv8 + OCR) to detect and read vehicle license plates.	✓	✓	High
FR-02	The system shall check vehicle access permission by comparing license plate against whitelist tables.	✓	✓	High
FR-03	The system shall communicate with IoT Gateway using HTTP to send OPEN or KEEP_CLOSED commands.	✓	✓	High
FR-04	The system shall store all images (snapshot and cropped plate) as binary data (BYTEA) in database.	✓	✓	High
FR-05	The system shall allow citizens to manage their personal vehicles (view, add new vehicles).	✓	✓	High
FR-06	The system shall allow citizens to register guests in advance with scheduled visit time.	✓	✓	High
FR-07	The system shall allow citizens to create emergency OTP codes (6 digits, valid for 15 minutes).	✓	✓	High
FR-08	The system shall allow security guards to monitor gate activities in real-time through dashboard.	✓	✓	High
FR-09	The system shall allow security guards to submit AI correction feedback when a plate is misrecognized, to improve recognition accuracy over time.	✓	✓	Medium
FR-10	The system shall allow citizens to view their personal vehicle access history (entry/exit logs filtered to their own vehicles).	✓	✓	Medium
FR-11	The system shall allow the AI Camera to automatically read a 6-digit OTP or QR UUID displayed on a smartphone screen and verify it without Guard interaction.	✓	✓	High
FR-12	The system shall allow citizens to generate a personal QR token (UUID, 3-minute validity) for non-motorized entry (pedestrians, cyclists).	✓	✓	Medium
FR-13	The system shall require security guards and managers to select a mandatory `action_reason` before executing any manual gate override.	✓	✓	High

Table 11: ... (continued)

ID	Requirement	Feasible	Testable	Important
FR-14	The system shall allow managers to approve or reject pending vehicle registrations and update requests (`pending_new`, `pending_update`).	✓	✓	High
FR-15	The system shall provide managers with a dashboard showing 4 KPIs: Total Traffic Volume, Automation Rate, Security Alerts, and Active Visitors.	✓	✓	High
FR-16	The system shall allow managers to create and list system user accounts.	✓	✓	Medium

4.2 Non-Functional Requirements

Table 12: Non-Functional Requirements

ID	Requirement	Feasible	Testable	Important
NFR-01	The AI recognition shall complete within 1000 milliseconds end-to-end latency.	✓	✓	High
NFR-02	The API response time shall be less than 200 milliseconds at 95th percentile.	✓	✓	High
NFR-03	The database query time shall be less than 100 milliseconds for complex queries.	✓	✓	High
NFR-04	The system shall support more than 100 concurrent users simultaneously.	✓	✓	High
NFR-05	The maximum image upload size shall be 10 megabytes per request.	✓	✓	Medium
NFR-06	The system shall use JWT authentication with 7-day expiry for security.	✓	✓	High
NFR-07	The system shall store passwords using bcrypt with salt rounds equal to 10.	✓	✓	High
NFR-08	The system shall implement Role-Based Access Control with three roles: citizen, guard, manager.	✓	✓	High
NFR-09	The system shall maintain 99.5% uptime availability during operation hours.	✓	✓	Medium
NFR-10	The OTP codes shall be 6 digits, unique, and expire after 15 minutes.	✓	✓	High

4.3 Data Requirements

- **User Data:** Stores credentials, profile information, and roles (Resident, Guard, Admin) for system access control.
- **Vehicle Data:** Records license plate numbers, vehicle types, and digital copies/images of registration documents for verification.
- **Access Control Lists:** Maintains a "Whitelist" for permanent residents and a "Temporary Whitelist" for pre-registered guests with specific expiration timestamps.
- **Authentication Tokens:** Stores short-lived OTP codes (15-minute validity) and dynamic QR codes for non-motorized entry (pedestrians/cyclists).
- **Activity Logs:** Detailed records of every entry/exit event, including timestamps, captured plate images, entry methods, and the specific actor involved.
- **AI Metadata:** Stores raw OCR output (`detected_text`) from each access event — including license plate text, OTP digits, or QR UUID strings — alongside image snapshots for audit purposes.
- **System Audit Trails:** Logs of manual overrides by security guards, including the mandatory justification/reason for opening the barrier.
- **Multimedia Storage:** Management of raw image inputs from cameras used for both real-time AI processing and historical security evidence.

4.4 Use Case Descriptions

1/ UC-01: Manage Personal Vehicles

Name	Manage Personal Vehicles
ID	UC-01
Actor	Primary: Citizen
Description	Citizen views, adds, edits, or deletes vehicles from personal whitelist.
Preconditions	• Citizen has logged in to the system • Citizen has valid account associated with a house
Postconditions	• **Success:** Vehicle is added or updated with is_active = false (waiting for approval), or vehicle is successfully deleted. • **Failure:** Error message displayed, changes not saved.
Basic Flow	1. Citizen opens the My Vehicles page 2. System displays list of current registered vehicles 3. Citizen selects Add New Vehicle, Edit Vehicle, or Delete Vehicle 4. Citizen enters/updates details (License Plate, Vehicle Type, Color) 5. System validates license plate format and checks for duplicates (ignoring current vehicle if editing) 6. System saves changes with is_active = false (pending approval), or deletes the vehicle 7. System shows success message to Citizen
Alternative Flow	**AF1: Invalid license plate format (at step 5)** 1. System detects invalid license plate format 2. System shows error message and asks Citizen to re-enter **AF2: License plate already exists (at step 5)** 1. System finds license plate already registered in database 2. System notifies Citizen that vehicle is already registered
Exception Flow	**EF1: Database connection failure** 1. System cannot connect to database 2. System logs error and displays friendly error message 3. Citizen asked to try again later

Table 13: UC_01: Manage Personal Vehicles

2/ UC-02: Create Emergency OTP Code

Name	Create Emergency OTP Code
ID	UC-02
Actor	Primary: Citizen
Description	Citizen creates a 6-digit OTP code for unexpected guests (walk-ins) calling from the gate.
Preconditions	• Citizen has logged in to the system • Citizen has valid account associated with a house
Postconditions	• **Success:** OTP code created with 15-minute validity period. OTP is displayed to Citizen and synced to Guard's monitor. • **Failure:** OTP not created, error message displayed.
Basic Flow	1. Guest arrives at gate and calls Citizen 2. Citizen selects Create OTP option on the app 3. System generates random 6-digit code (unique in active pool) 4. System saves OTP to access_tokens with valid_until = NOW() + 15 minutes 5. System displays OTP code on Citizen's app AND automatically synchronizes it to the Security Guard's monitor 6. Citizen reads or messages the OTP code to the guest
Alternative Flow	**AF1: OTP collision (at step 2)** 1. System generates code that already exists in active pool 2. System regenerates new unique code automatically 3. Flow continues normally
Exception Flow	**EF1: Database connection failure** 1. System cannot save OTP to database 2. System shows error message 3. Citizen asked to try again

Table 14: UC_02: Create Emergency OTP Code

3/ UC-03: Guard Verify OTP and Manual Override

Name	Guard Verify OTP and Manual Override
ID	UC-03
Actor	Primary: Security Guard, Manager
Description	Security Guard verifies OTP codes from guests and performs manual override actions (open barrier or keep closed) with mandatory reason selection. Managers can also perform manual overrides remotely.
Preconditions	• Actor has logged in to system • Guard is assigned to a specific gate, Manager is managing the zone • Dashboard is displaying gate monitoring view
Postconditions	• **Success:** OTP verified and consumed, barrier opens. Or manual action logged with reason, barrier state changed. Host information displayed for verification. • **Failure:** OTP invalid/expired/used, access denied. Error message shown.
Basic Flow	1. Guest reads the OTP code to the Security Guard (sent from Citizen) 2. Guard verifies the OTP against the synchronized code on their screen or enters it in Verify OTP field 3. System checks: OTP exists? Not used? Not expired? 4. If valid: System marks OTP as used and records used_at timestamp 5. System logs access with access_method = manual_otp 6. System sends OPEN command to IoT Gateway 7. System displays host information (name, apartment number) 8. Barrier opens automatically
Alternative Flow	**AF1: Manual Override - Open Barrier (at any time)** 1. Guard or Manager selects Manual Action: Open Barrier on the dashboard 2. System requires Actor to select reason from list: emergency_vehicle, vip_guest, ai_error, maintenance, other 3. If other selected, Actor must enter note 4. Actor confirms action 5. System logs action with access_method = manual_guard 6. System sends OPEN command to IoT Gateway **AF2: Manual Override - Keep Closed (at any time)** 1. Guard or Manager selects Manual Action: Keep Closed 2. System requires Actor to select reason: shipper_delivery, suspicious_vehicle, unauthorized, other 3. Actor confirms action 4. System logs action with is_access_granted = false 5. Barrier remains closed

Exception Flow	**EF1: OTP not found (at step 3)** 1. System cannot find OTP in database 2. System displays: OTP code is invalid 3. Guard informs guest, access denied **EF2: OTP already used (at step 3)** 1. System finds OTP but is_used = true 2. System displays: OTP code has already been used 3. Guard informs guest to request new code **EF3: OTP expired (at step 3)** 1. System finds OTP but valid_until < current time 2. System displays: OTP code has expired 3. Guard informs guest to request new code from host

Table 15: UC_03: Guard Verify OTP and Manual Override

4/ UC-04: AI-Powered Automatic Check-In

Name	AI-Powered Automatic Check-In
ID	UC-04
Actor	Primary: System (IoT Gateway + AI Service)
Description	When a vehicle approaches the gate, the IoT Gateway captures an image from the camera and sends it to the AI Service. The AI Service uses YOLOv8 to detect the license plate region and PaddleOCR to extract the plate text. The Backend applies a 3-step verification (permanent whitelist → temporary guest whitelist → deny). If authorized, the gate barrier opens automatically. All events are logged with timestamp and captured image.
Preconditions	• Camera and ultrasonic sensor at gate are operational • IoT Gateway (`TestingMode.py`) is running and connected to Backend • AI Service with YOLOv8 and PaddleOCR is available • Gate barrier hardware is functional • Vehicle database contains registered plates
Postconditions	• **Success:** Plate recognized and matched in whitelist (resident or pre-registered guest). Gate opens automatically. System logs event and toggles `is_inside` for resident vehicles. • **Failure:** Plate not recognized, confidence too low, or anti-passback violation. Gate remains closed. Guard dashboard alerted in real-time.

Basic Flow	1. Vehicle approaches gate and triggers IR proximity sensor 2. IoT Gateway detects sensor activation via Arduino serial communication 3. IoT Gateway reads the current frame from the OpenCV camera capture 4. IoT Gateway calls the local AI engine to extract the license plate text 5. AI Service preprocesses image (resize, normalize, enhance contrast) 6. AI Service runs YOLOv8 to detect the license plate bounding box 7. AI Service applies PaddleOCR to extract the plate text string 8. AI Service returns `plate_text` and confidence score 9. IoT Gateway sends `plate_text`, `lane_id`, and `image_base64` to Backend (`POST /api/v1/gates/check-in`) 10. Backend applies 3-step whitelist verification (Step 1: permanent, Step 2: guest, Step 3: deny) 11. Backend applies anti-passback check (`is_inside` flag) for resident vehicles 12. Backend creates `access_log` entry with `access_method = 'ai_plate_recognition'` 13. Backend updates the `is_inside` flag and stores the timestamp 14. Backend returns authorization response to IoT Gateway 15. IoT Gateway sends OPEN command to Arduino via serial port 16. Arduino activates servo motor to raise gate barrier 17. Vehicle passes through gate 18. IR sensor detects vehicle has passed 19. IoT Gateway sends CLOSE command to Arduino 20. Arduino lowers gate barrier

Alternative Flow	**AF1: Guest with pre-registered vehicle (Step 2 of 3-step flow)** 1. Backend does not find plate in `vehicles` table (Step 1 fails) 2. Backend queries `guest_registrations` for matching `guest_license_plate` with `status = 'approved'` 3. Backend validates that current time is within `[visit_start_time, visit_end_time]` 4. If valid: Backend logs event (storing `guest_reg_id`) and sends `OPEN` command **AF2: Low confidence requires confirmation (at step 10)** 1. AI Service returns confidence score below threshold (e.g., 85%) 2. IoT Gateway still sends plate to Backend for lookup 3. If partial match found, system alerts guard screen for confirmation 4. Guard reviews captured image and confirms or rejects 5. Flow continues based on guard decision **AF3: Multiple plates detected (at step 7)** 1. YOLOv8 detects multiple plate candidates in image 2. AI Service selects plate with highest confidence and largest area 3. AI Service processes selected plate for OCR 4. System logs all detected plates for review **AF4: Night mode with low light (at step 4)** 1. System detects low ambient light condition 2. IoT Gateway activates IR illuminator before capture 3. Camera captures image with IR enhancement 4. AI Service applies night-mode preprocessing filters

Exception Flow	**EF1: Camera capture fails (at step 4)** 1. Camera returns error or timeout on capture request 2. IoT Gateway retries capture up to 3 times 3. If all retries fail, system alerts guard for manual processing 4. Guard uses manual verification via intercom **EF2: AI Service unavailable (at step 5)** 1. IoT Gateway cannot connect to AI Service endpoint 2. System falls back to guard notification mode 3. Guard screen displays captured image for manual plate entry 4. Guard enters plate manually and system verifies against database **EF3: No plate detected in image (at step 7)** 1. YOLOv8 model returns empty detection result 2. IoT Gateway requests second image capture with adjusted settings 3. If second attempt fails, system alerts guard 4. Guard manually processes vehicle entry **EF4: Unregistered plate detected (at step 12)** 1. Backend finds no matching vehicle or guest registration 2. System logs unauthorized access attempt with captured image 3. Guard screen displays alert with vehicle image 4. Guard communicates with driver via intercom 5. Guard decides to allow manual entry or deny access **EF5: Gate barrier malfunction (at step 17)** 1. Arduino reports servo motor error or timeout 2. IoT Gateway logs hardware failure 3. System alerts maintenance team 4. Guard manually opens barrier using override key

Table 16: UC_04: AI-Powered Automatic Check-In

5/ UC-05: Guard Real-Time Gate Monitoring

Name	Guard Real-Time Gate Monitoring
ID	UC-05
Actor	Primary: Security Guard
Description	Guard monitors gate activities in real-time via the dashboard: live camera stream, recent access logs, and gate statistics.
Preconditions	• Guard has logged in to the system • Guard is assigned to a specific gate
Postconditions	• **Success:** Dashboard updates continuously with new events in real-time.
Basic Flow	1. Guard logs in → system redirects to the assigned gate dashboard 2. System displays: live camera stream, 20 most recent logs, hourly and daily stats 3. When a new vehicle event occurs → AI processes → Dashboard updates via WebSocket 4. Guard can click any log to view detail (image snapshot, plate text, access method, status)

Table 17: UC_05: Guard Real-Time Gate Monitoring

6/ UC-06: Automated OTP/QR Verification via Camera

Name	Automated OTP/QR Verification via Camera
ID	UC-06
Actor	Primary: AI Service / Guest or Citizen
Description	Guest holds their smartphone screen showing a 6-digit OTP (or resident shows a QR code) towards the Camera at the gate. The AI Service uses OCR to read the code automatically and sends it to the Backend for verification — no Guard interaction required.
Preconditions	• A valid OTP has been created (UC-03) or QR token issued • Camera and AI Service are operational
Postconditions	• **Success:** OTP/QR marked `is_used = true`, access log created, gate opens automatically. • **Failure:** Code invalid, expired, or unreadable — access denied, Guard notified.
Basic Flow	1. Guest/Citizen holds smartphone screen (OTP digits or QR) towards Camera 2. AI Service detects bright screen area in camera frame and applies OCR 3. Extracted code sent to Backend: `POST /api/v1/gates/verify-camera-otp` 4. Backend validates: code exists, `is_used = false`, within `valid_until` 5. Backend marks `is_used = true` → creates `access_log` → sends `OPEN` command 6. Gate opens automatically; Guard sees new log on real-time Dashboard
Alternative Flow	**AF1: OCR cannot read the screen (at step 2)** 1. Camera frame unclear (angle, glare, low resolution) 2. System displays guidance on gate LCD: adjust phone position 3. Guest repositions; flow retries from step 1
Exception Flow	**EF1: Code invalid or expired (at step 4)** 1. Backend finds no matching active token 2. Access denied; Guard dashboard receives alert 3. Guard assists guest manually (UC-03 or UC-07)

Table 18: UC_06: Automated OTP/QR Verification via Camera

7/ UC-08: Guard AI Correction Feedback

Name	Guard AI Correction Feedback
ID	UC-08
Actor	Primary: Security Guard
Description	When the AI misreads a license plate, the Guard corrects the text. The corrected value is stored in `access_logs.note` for audit and future reference.
Preconditions	• Guard has logged in • A recent access log exists with an incorrectly detected plate
Postconditions	• **Success:** Corrected plate text saved in `access_logs.note`.
Basic Flow	1. AI detects plate as ”51A-12345” with low confidence 2. Guard notices actual plate is ”51A-12346” 3. Guard clicks the log entry → selects ”Edit plate” 4. Guard enters correct plate text → submits 5. System updates `access_logs.note` with the corrected value

Table 19: UC_08: Guard AI Correction Feedback

8/ UC-09: Manager Vehicle Approval

Name	Manager Vehicle Approval
ID	UC-09
Actor	Primary: Manager
Description	Manager reviews and approves or rejects vehicle registration requests (`pending_new`) or update requests (`pending_update`) submitted by citizens.
Preconditions	• Manager has logged in • At least one vehicle with `pending_new` or `pending_update` status exists
Postconditions	• **Approve:** Vehicle status set to `approved`, `is_active = true`, citizen can use vehicle. • **Reject:** Vehicle reverted to previous state or removed; citizen notified.
Basic Flow	1. Manager navigates to "Pending Vehicles" page 2. System displays list of pending vehicles (`pending_new` and `pending_update`) 3. Manager clicks a vehicle to view details (plate, type, color, submitted changes) 4. Manager selects Approve or Reject 5. System updates vehicle status and records action in `system_audit_logs` 6. Manager dashboard refreshes to reflect updated queue

Table 20: UC_09: Manager Vehicle Approval

9/ UC-11: Manager Access Log Search

Name	Manager Access Log Search
ID	UC-11
Actor	Primary: Manager
Description	Manager searches, filters, and reviews detailed access logs across all gates in the managed zone.
Preconditions	• Manager has logged in
Postconditions	• **Success:** Filtered log list displayed with pagination; Manager can inspect individual entries including image snapshot.
Basic Flow	1. Manager navigates to "Access Logs" page 2. Manager applies filters: date range, gate/lane, access method, license plate, granted/denied 3. System queries `access_logs` and returns paginated results 4. Manager clicks a log entry to view detail: image snapshot, `detected_text`, `action_reason`, actor info

Table 21: UC_11: Manager Access Log Search

4.5 Manager Dashboard KPIs

The Manager Dashboard is designed to provide a concise, real-time operational overview of the residential area through four core Key Performance Indicators (KPIs), extracted directly from the `access_logs` table.

Table 22: Manager Dashboard KPIs

#	KPI	Description	Data Source (Logic)
1	Total Traffic Volume	Total number of entry/exit events recorded today (from 00:00 to current time).	Count all records in `access_logs` where date equals today.
2	Automation Rate	Percentage of gate openings triggered automatically by AI (plate recognition, camera OTP, or camera QR) out of total recorded events.	Ratio of records where `access_method` is `ai_plate_recognition`, `ai_camera_otp`, or `ai_camera_qr` to total records.
3	Security Alerts	Total number of access attempts denied — unregistered vehicle, invalid OTP, or anti-passback violation.	Count records where `vehicle_id`, `guest_reg_id`, and `token_id` are all `NULL` (no authorized entity matched).
4	Active Visitors	Number of pre-registered guest vehicles currently inside the compound (inbound entries minus outbound exits).	Count inbound logs linked to `guest_registrations` minus outbound logs for the same guest registrations.

5 IoT Gateway Implementation (Python)

The IoT Gateway is a Python application (`TestingMode.py`) that serves as the bridge between the Backend server and the Arduino hardware. It connects to the Backend via WebSocket (Socket.IO) and communicates with Arduino via PySerial to control the gate barrier.

IoT Gateway Architecture

1. **Configuration & Initialization**:
   - Establishes serial connection to Arduino via PySerial at 9600 baud.
   - Connects to Backend WebSocket server as Socket.IO client on port 5000.
   - Initializes OpenCV camera capture (`cv2.VideoCapture`).
2. **Sensor Detection Handler**:
   - Arduino sends `CAR_ARRIVED` signal via serial when HC-SR04 sensor detects a vehicle at the gate.
   - Gateway reads serial data continuously in the main loop.
   - Upon `CAR_ARRIVED` signal, Gateway reads the current camera frame and initiates AI recognition.
3. **Image Capture and AI Processing**:
   - Reads current camera frame using OpenCV (`cv2.VideoCapture`).
   - Encodes frame to base64 format for HTTP transmission.
   - Calls local AI engine (`extract_license_plates`) to detect and extract plate text.
4. **Access Verification**:
   - Sends `POST /api/v1/gates/check-in` to Backend with `lane_id`, `plate_text`, and `image_base64`.
   - Backend returns authorization response with `action` field: `OPEN` or `KEEP_CLOSED`.
   - Gateway processes response and sends corresponding serial command to Arduino.
5. **Gate Control via Serial**:
   - On `OPEN`: sends `ENTRY_GO:OwnerName\n` to Arduino → Arduino opens barrier and shows welcome message on LCD.
   - On denial: sends `DENY_PLATE:PlateText\n` → Arduino shows `UNAUTHORIZED GUEST` on LCD.
   - Arduino auto-closes barrier when exit HC-SR04 sensor detects vehicle has passed through.
6. **WebSocket Event Handling**:
   - Continuously streams camera frames to Frontend via `video_stream` Socket.IO event.
   - Emits `scan_result` event to Frontend with plate, owner name, and vehicle type after each check-in.
   - Listens for `manual_command` event from Backend: `OPEN` → sends `ENTRY_GO:Operator\n`; `CLOSE` → sends `FORCE_CLOSE\n`.

5.1 Serial Communication Protocol

The following tables define the serial protocol between the IoT Gateway (`TestingMode.py`) and the Arduino (`Hardware.ino`) operating at 9600 baud.

Commands sent from IoT Gateway to Arduino:

Table 23: Serial Commands: IoT Gateway → Arduino

Command	Meaning
`ENTRY_GO:`*OwnerName*`\n`	Open barrier; Arduino displays welcome message with owner name on LCD and beeps once.
`FORCE_CLOSE\n`	Emergency close all barriers; Arduino displays `EMERGENCY CLOSE` on LCD and beeps three times.
`DENY_PLATE:`*PlateText*`\n`	Access denied; Arduino displays `UNAUTHORIZED GUEST` on LCD and beeps once.

Signals received by IoT Gateway from Arduino:

Table 24: Serial Signals: Arduino → IoT Gateway

Signal	Meaning
`CAR_ARRIVED\n`	HC-SR04 ultrasonic sensor detected a vehicle approaching the gate; triggers AI license plate recognition in the Gateway.

5.2 Analysis of Problems & Solutions

To ensure the system not only works but also performs well and securely, the following practical technical issues were analyzed and resolved:

1/ Query Optimization and Indexing

- **Problem:** As the number of access logs grows, querying for vehicle verification and displaying access history can become slow.
- **Solution:** Indexes were created on **license_plate** in vehicles table and **log_timestamp** in access_logs table. The B-tree index on license plate enables instant plate lookup during AI check-in process.

2/ Image Storage Strategy

- **Problem:** Each access event captures both full snapshot and cropped plate images, leading to large storage requirements.
- **Solution:** PostgreSQL BYTEA column type is used for storing images directly in the database, ensuring transactional consistency. Images are compressed before storage. Old access logs (older than 90 days) are archived to separate storage.

3/ Real-time Communication

- **Problem:** The guard dashboard needs to display access events in real-time without constant polling.

- **Solution:** WebSocket connection is established between Backend and guard dashboard. When new access event occurs, Backend broadcasts the event to all connected guard screens instantly.

4/ Hardware Failure Handling

- **Problem:** If the camera, AI service, or Arduino fails, the gate should not block vehicle access indefinitely.
- **Solution:** The IoT Gateway implements a fallback mechanism. If AI service is unavailable for more than 30 seconds, system switches to manual mode and alerts guard. Guard can use manual override button to control the barrier directly.

5/ Security and Authentication

- **Problem:** The IoT Gateway API endpoints could be exploited if not properly secured.
- **Solution:** All API requests between Backend and IoT Gateway use API key authentication. The API key is stored in environment variables and validated on each request. Additionally, the IoT Gateway only accepts connections from whitelisted IP addresses.

6 Database

image/Database.png

Figure 3: Database Schema Diagram

6.1 Database Schema Tables

- **ZONES** (zone_id, zone_name, description, created_at)
  - Primary key: zone_id
- **GATES** (gate_id, zone_id, gate_name, is_active)
  - Primary key: gate_id

  - Foreign key: zone_id references ZONES(zone_id)
- **LANES** (lane_id, gate_id, lane_name, direction_enum)
  - Primary key: lane_id
  - Foreign key: gate_id references GATES(gate_id)
- **IOT_DEVICES** (device_id, lane_id, device_name, device_type, ip_address, mac_address, status, last_ping)
  - Primary key: device_id
  - Foreign key: lane_id references LANES(lane_id)
- **USERS** (user_id, username, password_hash, full_name, email, role, avatar_url)
  - Primary key: user_id
- **CITIZENS** (user_id, address, zone_id, phone_number, identity_card_number, is_house_owner)
  - Primary key: user_id
  - Foreign keys: user_id references USERS(user_id), zone_id references ZONES(zone_id)
- **SECURITY_GUARDS** (user_id, assigned_gate_id, employee_code, shift_start, shift_end)
  - Primary key: user_id
  - Foreign keys: user_id references USERS(user_id), assigned_gate_id references GATES(gate_id)
- **MANAGERS** (user_id, managed_zone_id, department_name)
  - Primary key: user_id
  - Foreign keys: user_id references USERS(user_id), managed_zone_id references ZONES(zone_id)
- **VEHICLES** (vehicle_id, owner_user_id, vehicle_type, license_plate, vehicle_color, is_inside, is_active, last_log_time, registered_at)
  - Primary key: vehicle_id
  - Foreign key: owner_user_id references USERS(user_id)
- **GUEST_REGISTRATIONS** (registration_id, host_user_id, guest_name, vehicle_type, guest_license_plate, visit_start_time, visit_end_time, status)
  - Primary key: registration_id
  - Foreign key: host_user_id references USERS(user_id)
- **ACCESS_TOKENS** (token_id, issued_by, token_data, valid_from, valid_until, is_used, used_at)
  - Primary key: token_id
  - Foreign key: issued_by references CITIZENS(user_id)
- **ACCESS_LOGS** (log_id, lane_id, vehicle_id, guest_reg_id, token_id, guard_id, check_in_time, detected_text, access_method, action_reason, note, image_snapshot_data)
  - Primary key: log_id

  - Foreign keys: lane_id references LANES(lane_id), vehicle_id references VEHICLES(vehicle_id), guest_reg_id references GUEST_REGISTRATIONS(registration_id), token_id references ACCESS_TOKENS(token_id), guard_id references SECURITY_GUARDS(user_id)

- **SYSTEM_AUDIT_LOGS** (audit_id, actor_id, device_id, action_type, target_table, target_id, action_details, performed_at)
  - Primary key: audit_id
  - Foreign keys: actor_id references USERS(user_id), device_id references IOT_DEVICES(device_id)